Gradient Descent in Neural Networks | Batch vs Stochastic vs Mini-Batch Gradient Descent

➢ Introduction

In this session, we'll explore **how neural networks learn** by minimizing errors using **Gradient Descent** — the backbone of most machine learning algorithms.
We'll also compare **Batch**, **Stochastic**, and **Mini-Batch Gradient Descent**, and understand which is better, faster, and when to use each.

➢ Gradient Descent

Concept:

Gradient Descent is an optimization algorithm used to **minimize the loss function** by updating the model's parameters (weights and biases).

Formula:

$$w := w - \eta \frac{\partial L}{\partial w}$$

Where:

- w: weight
- η: learning rate (step size)
- $\frac{\partial L}{\partial w}$: gradient of loss function w.r.t weight

This process repeats iteratively until convergence (minimum loss).

Intuition:

Think of it like **rolling a ball down a hill**:

- The ball moves in the direction of the **steepest descent** (negative gradient).
- The goal: reach the lowest point of the valley (minimum loss).

➢ Backpropagation Algorithm

Backpropagation is how the network **computes the gradients** for all weights efficiently.

1. **Forward Pass** → Compute predicted output and loss.
2. **Backward Pass** → Compute gradients using the chain rule.
3. **Weight Update** → Adjust weights using gradient descent.

So, **backpropagation finds gradients**, and **gradient descent updates the weights** using those gradients.

➢ Batch Gradient Descent

Definition:

In **Batch Gradient Descent**, the **entire training dataset** is used to compute the gradient and update the weights **once per epoch**.

Formula:

$$w := w - \eta \frac{1}{N} \sum_{i=1}^{N} \frac{\partial L_i}{\partial w}$$

Where (N) is the total number of samples.

Characteristics:

- Uses the full dataset per update.
- Stable convergence.
- Computationally heavy for large datasets.
- Best for smaller datasets.

Pros:

- Smooth convergence to the global minimum (for convex problems).
- More accurate gradient estimate.

Cons:

- Slow when dataset is large.
- High memory usage.

➢ Questions / Differences

Here's a summary table for quick comparison:

Type	Data per update	Speed	Convergence	Noise
Batch GD	All samples	Slow	Stable	None
Stochastic GD (SGD)	1 sample	Fast	Noisy	High
Mini-Batch GD	Subset (batch)	Moderate	Balanced	Medium

➢ Code Demo (Batch Example)

Batch Gradient Descent Example

```
for epoch in range(epochs):
  y_pred = model(X)
  loss = criterion(y_pred, y_true)
  loss.backward()
  optimizer.step()
  optimizer.zero_grad()
```

All training samples are used in each weight update.

➢ Which is faster to converge?

- **Batch GD** → Slower updates but stable path to minimum.
- **Stochastic GD** → Faster updates but highly noisy; oscillates around the minimum.

- **Mini-Batch GD** → **Best of both worlds** — faster and smoother convergence.

➢ Stochastic Gradient Descent (SGD)

Definition:

In **SGD**, the model updates weights **after each training sample**.

$$w := w - \eta \frac{\partial L_i}{\partial w}$$

Characteristics:

- Updates weights immediately after each sample.
- Very fast but noisy.
- Can escape local minima due to randomness.

Pros:

- Efficient for large datasets.
- Often faster convergence in practice.

Cons:

- High variance (loss fluctuates).
- Harder to reach exact minimum.

➢ Vectorization

Vectorization means using **matrix operations** instead of loops to make computations faster.

Frameworks like **NumPy**, **TensorFlow**, and **PyTorch** are optimized for vectorized operations, utilizing CPU/GPU efficiently.

Example:

Instead of looping

y = w * x + b # Vectorized, faster

➢ Mini-Batch Gradient Descent

Definition:

Mini-Batch Gradient Descent splits the dataset into small batches (e.g., 32, 64, 128 samples), and updates weights for each batch.

$$w := w - \eta \frac{1}{m} \sum_{i=1}^{m} \frac{\partial L_i}{\partial w}$$

Pros:

- Faster than batch GD.
- Smoother than SGD.
- Enables vectorized computation.

Most commonly used in modern deep learning.

➢ Why batch size is provided in multiple of 2

Because hardware (especially GPUs) operates optimally with **powers of 2** (e.g., 32, 64, 128, 256).

It aligns with memory blocks and vectorized computation units, making training **faster and more efficient**.

➢ What if batch size doesn't divide number of rows?

If the total number of samples is not divisible by the batch size:

- The **last batch** will simply have **fewer samples**.
- Most deep learning frameworks (PyTorch, TensorFlow) handle this automatically.

Example:

If 103 samples and batch size = 32 → batches = [32, 32, 32, 7].

➢ Code Demo

```
for epoch in range(epochs):
  for X_batch, y_batch in dataloader:
    y_pred = model(X_batch)
    loss = criterion(y_pred, y_batch)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

Each batch contributes to weight updates separately.

➢ Outro

✓ Key Takeaways:

Type	Uses	Best For
Batch GD	Whole dataset per step	Small datasets
SGD	1 sample per step	Large, online learning
Mini-Batch GD	Subset per step	Deep learning (most common)

Mini-Batch Gradient Descent is the *default standard* for training deep neural networks, offering a balance of speed, stability, and accuracy.